FIT3171 Databases

Assignment 2 - Creating, Populating and Manipulating Databases

BigRig Movers (BRM)

Purpose	Students will be asked to implement, via SQL, a small database in the Oracle RDBMS from a provided logical model case study, followed by the insert of appropriate data into the created tables. Once populated, the database will perform specified DML commands and make specified changes to the database structure via SQL and code PL/SQL to enforce business rules. Students will then use SQL and NoSQL to write queries to produce the specified output. This task covers learning outcomes: 1. Apply the theories of the relational database model. 3. Implement a relational database based on a sound database design. 4. Manage data that meets user requirements, including queries and transactions. 5. Contrast the differences between non-relational database models and the relational database model. 6. Develop programming structures within a database backend.
Your task	**This is an open-book, individual assessment.** The output for this assessment will be a set of tables and data implemented in the Oracle RDBMS. In addition, students will create a set of relational (Oracle) and non-relational (MongoDB) queries that meet the user requirements and code PL/SQL to enforce business rules.
Value	**35%** of your total marks for the unit (30 mks Assignment Task, 5 mks Supporting Applied tasks)
Due Date	**Applied Tasks due: Monday of the week following the Applied Session at 9 am** **Assignment Tasks due: Monday, 8 June 2026 at 11:55 pm**
Submission	• Via Moodle Assignment Submission • **FIT GitLab check-ins will be used to assess the history of development** • **An interview may be required to validate the submission. Based on this interview, the graded mark may be adjusted.**
Assessment Criteria	• Complete the Supporting Applied Tasks • Application of relational database principles. • Handling of transactions and the setting of appropriate transaction boundaries. • Application of SQL statements and constructs to create and alter tables, including the required constraints and column comments, populate tables, modify existing data in tables, and modify the "live" database structure to meet the expressed requirements (including appropriate use of constraints). • Application of relational algebra operations to produce outputs that meet user requirements • Application of SQL select statements to produce outputs that meet user

	requirements. ● Mapping of relational database data into a non-relational database data structure. ● Application of MongoDB operations to produce outputs that meet user requirements. ● Development of Procedures and Triggers (PL/SQL) to enforce business rules and data integrity
Late Penalties	● 5% of the marks available for the task (-5 marks) deduction per calendar day or part thereof for up to one week ● Submissions over 7 calendar days after the due date will receive a mark of zero (0), and no assessment feedback will be provided.
Support Resources	See Moodle Assessment page
Feedback	Feedback will be provided on student work via: ● general cohort performance ● specific student feedback fifteen working days post-submission ● a sample solution

SCENARIO

Your task for this assignment is to design a model for a database that supports the activities of a trucking business called BigRig Movers (BRM).

BigRig Movers is a company which provides prime movers (trucks) and trailers to allow customers to move items from one location to another.

https://www.nhvr.gov.au/C2021G00386

Each truck owned by the business is identified by its Vehicle Identification Number (VIN). The truck's registration number, total kilometres travelled, and the kilometre reading of the last service are recorded.

Each trailer owned by the business is identified by a five-character trailer code, e.g., REF08.

For both trucks and trailers, BigRig Movers records the date the business purchased the equipment and the purchase price. Trucks and trailers are purchased separately.

Trucks and trailers are paired together in different ways depending on the customer's requirements. Each such pairing is referred to as a combination.

The company has several employees with roles of Manager (B), Truck Dispatcher (T), Mechanic (M), or Driver (D) - each employee has only one role. Employees are identified by an employee number. Each employee who is not a manager is assigned to one of the company managers to create a reporting line; all managers report to the business owner. Details are kept of the employee's name (family and given) and contact number. If the employee is a driver, their licence number is also recorded.

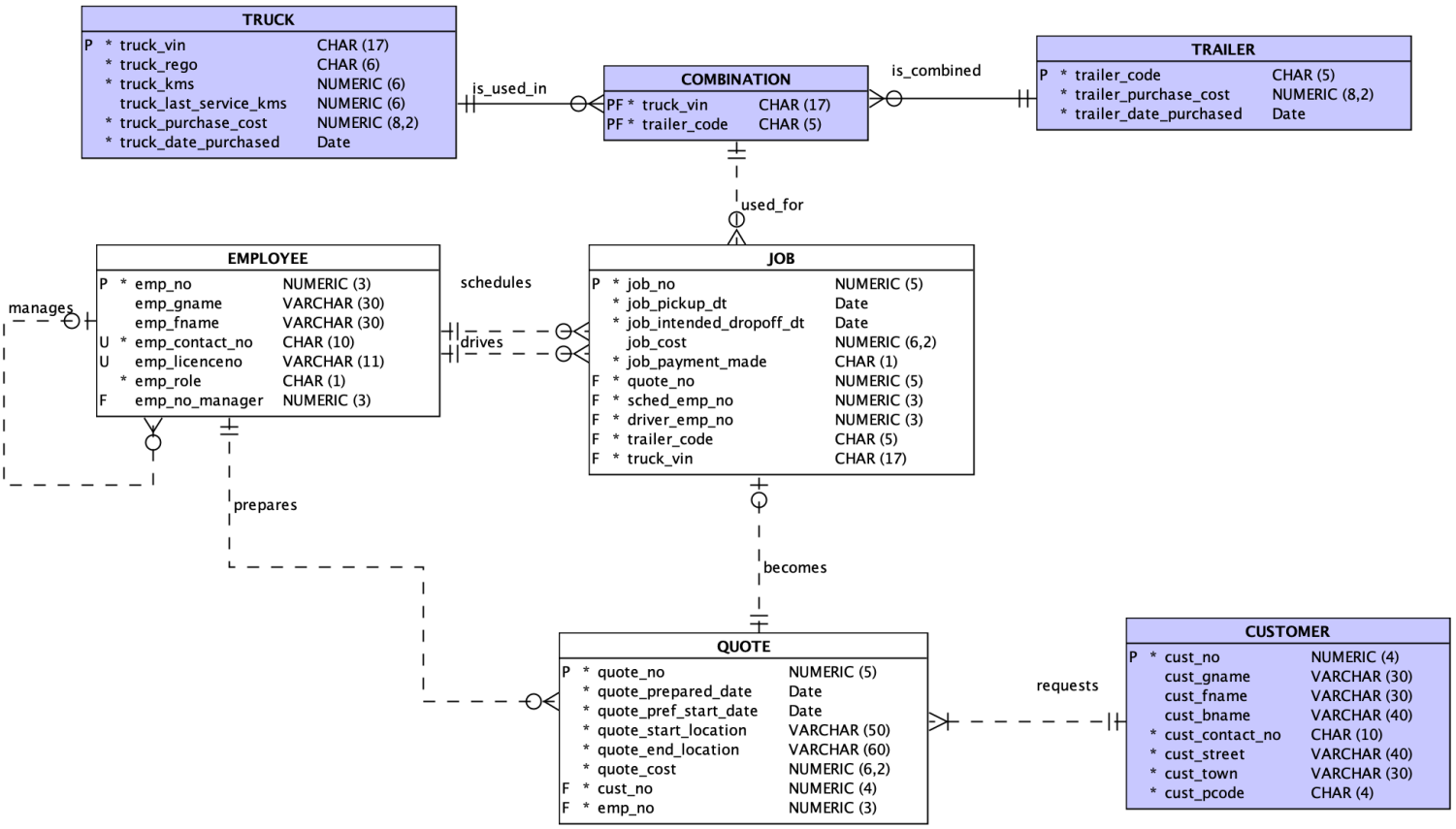
The Truck Dispatcher is responsible for managing the booking and assignment of the vehicles. When a customer calls up to get a quote, the Truck Dispatcher will first ask for the unique customer number. If this is the first time the customer is making a booking with BigRig Movers, the Truck Dispatcher will ask for the customer's details for registration purposes. These details include customer name (family and given), business name (if applicable), customer address and contact number. When getting a quote, the customer will indicate the preferred booking start date, the pickup location, and the drop-off location. Based on their knowledge of the trucks and trailers, the Truck Dispatcher will calculate a price for the quote. Note that, at the quote stage, no truck or trailer is assigned. The Truck Dispatcher

records the date on which the quote was prepared and the quoted cost. The Truck Dispatcher who prepared the quote must be recorded.

The customer may decide to accept the hire quote immediately, or may call back later with a decision. If the customer calls back later, they will need to provide the quote number previously given by the Truck Dispatcher. Only when the customer has agreed on the quote costing will the Truck Dispatcher assign the quote to a job (schedule the job). If the customer does not call back with a decision, it is regarded as an unfulfilled quote and left unfulfilled on the system.

Each job is assigned a unique job number. When the quote is assigned to a job, the Truck Dispatcher will negotiate the actual pickup date/time and an intended drop-off date/time with the customer based on the availability of a suitable truck and trailer. The selected truck and trailer combination will be assigned to the job. A driver is then scheduled to drive. The Truck Dispatcher also calculates the final job cost, which may be different from the quoted price. When this occurs, the original quoted price is not updated, a revised job costing is, however, recorded. This revised job costing is only recorded where the job cost is different from the quoted cost. A record is also kept noting if the customer has paid for the job. The Truck Dispatcher who assigned the quote to a job must be recorded (note that this may be a different dispatcher from the one who prepared the quote).

Based on these requirements, a data model has been created for BRM:



You must ensure that any activities you carry out in the database to complete the assignment tasks conform to the requirements of the model displayed above.

The schema/insert file for creating this model (brm-schema-insert.sql) is available in the archive ass2_student.zip. This file partially creates the BigRig Movers tables and populates several of them (those shown in purple on the supplied model above). Please read this schema carefully and be sure you understand the various data requirements.

IMPORTANT points for you to observe when completing this assignment are:

1. The ass2-student.zip archive also contains seven script files to code your answers in. **You MUST ensure these files are regularly pushed to the GitLab server so that a clear development history is available for the marker to verify your work (a minimum of fourteen pushes are required - 2 pushes per file).** In each file, you ***must*** fill in the header details with your name and student ID before beginning to work. Your SQL script files **must not include any SPOOL or ECHO commands**. Although you might include such commands when testing your work, **they must be removed before submission** (a -10 mark grade penalty will be applied if your documents contain spool or echo commands).
2. All work for this assignment must be completed in VS Code, you must not use ORDS for any part of the task.
3. You are free to make assumptions if needed. However, ***your assumptions must align with the details here and in the Ed Assignment 2 forum*** and must be clearly documented (see the required submission files).
4. Table names used in all tasks **must not be prefaced with a username**, i.e. you must use, for example, CUSTOMER **NOT** abc001.CUSTOMER (where abc001 represents your authcate).
5. When handling dates with SQL, the default date format must not be assumed; you must use ***the TO_DATE and TO_CHAR functions where appropriate***.
6. **ANSI joins must be used** where the joining of tables is required.
7. In completing this assignment, you **must not** use an approach which involves disabling foreign key constraints during any part of your solution.
8. In completing the following tasks, you must ***design your test data so that you always get output for the queries specified below*** - this may require you to add further data as you move through completing the required tasks. Such extra data MUST be added as part of Task 2 (i.e. as part of your test data load). ***Correct queries that do not produce any output ("no rows selected" message) using your test data will lose 50% of the marks allocated***. So, you should carefully check your test data and ensure it thoroughly validates your SQL queries.
9. You are **required to include clear, *concise*, and informative comments throughout your SQL, PL/SQL (Task 5 only) and MongoDB code**. Comments should describe the purpose, logic, and key steps of your approach. Insufficient or missing comments will result in a loss of marks (see the marking guide at the end of this document).
10. **If you are doing your development on Docker, you *must* validate that your code will work on 19c by running it against your Monash Oracle Account before submission (your work will be graded using the Monash Oracle Database).**

Steps for working on Assignment 2

1. Download the Assignment 2 Required Files zip archive (ass2-student.zip) from Moodle.
2. Extract the zip archive and place the contained files in your local repository in the folder:
   /Assignments/Ass2
   Do not add the zip archive to your local repo.
3. Examine the extracted files, i.e. read carefully through them and ensure you understand their content.
4. In each supplied script, fill in the header details with your name and student ID. Then, add, commit and push them to the FIT GitLab server.
5. Run brm-schema-insert.sql from the supplied zip archive to set up the initial state of the database.
6. Write your answer for each task in its respective file (e.g. write your answer for task 1 in T1-brm-schema.sql and so on).
7. Save, add, commit and push the file/s **regularly** while working on the assignment.
8. Finally, when you have completed all tasks, separately run each SQL, PL/SQL (Task 5 only), or MongoDB ***as a script (not as individual statements) and ensure there are no errors***. Upload all required files from your local repository to Moodle. Check that the files you have uploaded are the correct files (download them from Moodle into a temporary folder and check that they are correct). After you are sure they are correct, submit your assignment.

****

For all assignment tasks, **your final script must run as a script without errors, except for SQL errors generated by the DROP TABLE/DROP SEQUENCE/RAISE_APPLICATION_ERROR statements**. **Any task script that runs with an error will receive a maximum grade of half the task's available marks minus 1**. For example, if your task 1 script runs with an error, regardless of the code contained, your maximum grade will be 15/2 => 7.5 - 1 = 6.5 marks. This will be applied even if the error is simply a forgotten semicolon. Thus, **please carefully verify that your final scripts for all tasks run without error**.

In arriving at your solutions for Assignment 2 you are ONLY permitted to use the SQL/NoSQL structures and syntax that have been covered within this unit:

- **Topic 5 Workshop and Applied 6 - Creating & Populating the Database**
- **Topic 6 Workshop and Applied 7 - Insert, Update, Delete (DML) and Transaction Management**
- **Topic 7 Workshop and Applied 8 - SQL Part I - Basic - Intermediate**
- **Topic 8 Workshop and Applied 9 - SQL Part II - Advanced**
- **Topic 9 Workshop and Applied 10 - PL/SQL 1**
- **Topic 10 Workshop and Applied 11 - PL/SQL 2**
- **Topic 12 Workshop and Applied 12 - Non-Relational Database**

SQL/NoSQL syntax and commands outside the covered work will NOT be accepted/marked.

You must NOT use PL/SQL commands such as `BEGIN … END` other than in Task 5; nor SQL structures such as `WITH … SELECT…` since these were not covered in the unit. Views must not be used in completing these tasks.

You must also keep up to date with the Ed Assignment 2 forum, where further clarifications may be posted. Please **ensure you do not publicly post anything that includes your reasoning, logic, or any part of your work to this forum**; doing *so violates Monash's plagiarism/collusion rules* and carries significant academic penalties. Attend a consultation session or use a private Ed post to raise such questions.

****

GIT STORAGE

Your work for these tasks MUST be saved in your local working directory (repo) in the Assignments/Ass2 folder and **regularly pushed to the FIT GitLab server to build a clear history of the development of your approach**. ***Any work outside this folder will not be marked***. A minimum of fourteen pushes to the FIT GitLab server is required (2 pushes per solution script). Please note that fourteen pushes are a minimum; we expect significantly more in practice. All commits must include a **meaningful commit message** that clearly describes what the particular commit is about and **must be correctly assigned to your valid GitLab author**.

You must regularly check that your pushes have been successful by logging in to the FIT GitLab server's web interface; you must not simply assume they are working. Before submission via Moodle, you must log in to the GitLab server's web interface and ensure your submission files are present and their names are unchanged.

****

TASK 1: DDL (15 marks)

For this task, you are required to add to **T1-brm-schema.sql**, the CREATE TABLE and CONSTRAINT definitions that are missing from the supplied partial schema script in the positions indicated by the comments in the script. **You must use the default delete rule (i.e. no action/restrict) for all foreign keys**.

The table below provides details of the meaning of the attributes in the three missing tables. You **MUST use the same relation and attribute names and data types/sizes as shown in the data model** above to name the tables and attributes that you add. The attributes **must be in the same order as shown in the model**. **In addition to the listed attributes below, you must ensure that any foreign keys (as shown in the model) are also added**. These new DDL commands *must be hand-coded, not generated in any manner (generated code will not be marked)*.

Table name	Attribute name	Meaning
EMPLOYEE		
	emp_no	Employee number
	emp_gname	Employee given name
	emp_fname	Employee family name
	emp_contact_no	Employee contact number
	emp_licenceno	Employee licence number (only for drivers)
	emp_role	Employee role - Manager (B), Truck Dispatcher (T), Mechanic (M), or Driver (D)
QUOTE		
	quote_no	Quote number
	quote_prepared_date	The date the quote was prepared
	quote_pref_start_date	The preferred start date for the quote
	quote_start_location	The start location for the quote
	quote_end_location	The end location (destination) for the quote
	quote_cost	The quoted cost
JOB		
	job_no	Job number
	job_pickup_dt	Job scheduled pick up date and time
	job_intended_dropoff_dt	Job intended drop-off date and time
	job_cost	The actual job cost (if different from the quote cost)
	job_payment_made	Flag to note whether the job has been paid (Y or N)

To test your code, you must first run the provided script brm-schema-insert.sql to create the purple supplied tables. Then, run your T1-brm-schema.sql.

TASK 2: Populate Sample Data (10 marks)

Before proceeding with Task 2, you must ensure you have successfully run the file brm-schema-insert.sql (which **must not be edited in any way**) followed by the extra definitions that you added in Task 1 above (T1-brm-schema.sql).

Load the EMPLOYEE, QUOTE and JOB tables with **your own test data** using the supplied **T2-brm-insert.sql** script file, and SQL commands which will insert *as a minimum*, the following sample data:

- 10 EMPLOYEE entries
  - Involved at least 2 managers, 2 truck dispatchers, 1 mechanic and 2 drivers
- 30 QUOTE entries
  - Involved at least 5 different customers and 2 truck dispatchers
  - Involved at least 2 customers who place at minimum 2 quotes
- 20 JOB entries
  - Involved at least 10 truck/trailer combinations
  - Involved at least 5 truck/trailer combinations that are used in at least 2 jobs
  - Involved at least 2 quotes that have never been placed as job
  - Involved at least 5 jobs that have slightly higher or lower cost than the quote
  - Involved at least 5 jobs that have the same cost as the quote

In adding this data, you must ensure that the test data thoroughly tests the model as supplied to ensure your schema is correct (*you are not required to submit or code fail tests; all insert statements must execute correctly*).

Your inserted data must conform to the following rules:

(i) Treat all the data you add as a single transaction, since you are setting up the initial test state for the database.

(ii) The primary key values for this data should be hardcoded values (i.e. NOT make use of sequences). If the primary key attribute's data type is numeric, it must consist of values below 100.

(iii) Dates used must be chosen between the 1st May 2026 and 31st July 2026.

(iv) The data added must be sensible (e.g. the drop-off time must be after the pick up time; only employees with the role Driver can drive the truck; the job must align with the quote; etc).

For Task 2 **ONLY**, you may manually look up/calculate and include values for the loaded tables/data directly where required. However, you can still use SQL to get any non-key values if you wish. You may make use of AI & Generative AI tools to assist you with the generation of this data; however, if you do so, you must clearly acknowledge the source following Monash Guidelines. Place the acknowledgement in the "AI and GenAI Acknowledgement and Prompts" section at the top of T2-brm-insert.sql.

Note that you can only use AI & GenAI tools for Task 2; you must NOT use them for other tasks.

In carrying out Task 2, you must not modify any data or add any further data to the tables populated by the brm-schema-insert.sql script. Design your test data to get output for the SQL scripts/queries specified below - this may require you to add further data as you complete the required tasks.

TASK 3: DML (15 marks)

Your DML commands for this task (Task 3) must be placed in the supplied SQL Script **T3-brm-dm.sql**

For this and *all subsequent Tasks*, **you are NOT permitted to**:

- manually look up a value in the database, obtain its primary key, or manually obtain the highest/lowest value in a column,
- manually calculate values external to the database, e.g. on a calculator, and then use such values in your answers. ***Any necessary calculations must be carried out as part of your SQL code*** or
- assume any particular contents in the database - rows in a table are potentially in a constant state of change
- make use of the ALTER TABLE … DISABLE CONSTRAINT … SQL command to disable a foreign key

Your answers must recognise that you are dealing with only a small sample snapshot of a multiuser database; as such, you must operate on the basis that there will be ***more data in all the database tables than you currently have access to. Thus, data will be in a constant state of change. Your answers must work regardless of the extra quantity of this extra "real" data and the fact that multiple users will operate in the tables simultaneously. You must consider this aspect when writing SQL statements.***

For any following SQL tasks, **your SQL must correctly manage transactions and use sequences to generate new primary keys for numeric primary key values** (a new primary key value cannot be hardcoded as a number or value).

You must ONLY use the data provided in the questions' text.

For task 3 you are required to complete the following sub-tasks in the same order they are listed. Where you have been supplied with a string contained in *italics*, such as *Flintstone Store* you may search in the database using the string as listed. Where a particular case (upper case, lower case, etc.) for a word is provided, you must only use that case within your SQL code (you may modify the case via a function call in your code). When a name is supplied, you may break the name into given name and family name, for example, *Aurello Brown* can be split into Aurello and Brown, again note that the case must be maintained as it was supplied.

(a) Oracle sequences will be implemented in the database for the subsequent insert of records for the EMPLOYEE, QUOTE and JOB tables.
Provide the CREATE SEQUENCE statements to create sequences which could be used to provide primary key values for the EMPLOYEE, QUOTE and JOB tables. These sequences must start at 300 and increment by 5. Immediately before the create sequence commands, place an appropriate DROP SEQUENCE commands so that they will cause the sequences to be dropped before being created if they exist. Please note that there can only be three sequences introduced and used in Task 3.

[1 mark]

Before working on the questions below, you need to check your database:

> If you do not have a manager named *Sarah Mitchell* and a driver named *Michael Johnson* in your current database, you must go back to Task 2 and add these additional manager and driver data.

Re-run Task 1 and Task 2 files, then back to this task.

Question 3b, 3c, 3d and 3e are related questions. You can use the information provided below as needed in any part of task 3.

(b) On 15 May 2026, a new employee named *Aurello Brown* (contact number: *0431952053*) joined the BigRig Move as a Truck Dispatcher (*T*) and was assigned to a Manager (*B*) named *Sarah Mitchell*. Take the necessary steps in the database to record this new employee. You may assume there is only one manager with such a name. You may make up your own values for other required attributes.

[2 marks]

(c) On 17 May 2026, Aurello Brown prepared a quote for an existing customer named *VICTORIA ELLA* who works for the business *Flintstone Store*. The quote is for transporting garden statues from *29 Kuranda Road, Adelaide SA 5030* to *9 Albatros Drive, Mount Gambier SA 5270* on 25 May 2026. The quoted cost is $1000.

VICTORIA agreed to the quote, paid the cost (*Y*), and Aurello assigned a job to this quote. The pickup time is set for 9 AM on 25 May 2026. The driving duration between these two cities is predicted to be 5 hours long. A driver (*D*) named *Michael Johnson* is assigned to this job. The truck with VIN *1HGBH41JXMN109186* and the trailer with trailer code *TRL08* are scheduled for this job.

Take the necessary steps in the database to record the required entries. You can assume there is only one customer named *VICTORIA ELLA* from the *Flintstone* Store, and only one driver named Michael Johnson in the database. You must treat all the changes as a single transaction.

[6 marks]

(d) On 20 May 2026, *VICTORIA ELLA* called BRM and asked to shift the pickup time from 9 AM to 2 PM on 25 May 2026. Aurello recalculated the cost and the job cost is 20% higher than the previously quoted cost. She (VICTORIA) agreed to this new cost and paid the gap. Make these required changes to the data in the database. You may assume that she only received one quote on 17 May 2026.

[4 marks]

(e) On 23 May 2026, *VICTORIA ELLA* from the *Flintstone Store* called BRM again to cancel the job due to production issues. Please take the necessary steps in the database to remove the job from the system. You may assume that she (VICTORIA) only places one quote on 17 May 2026.

[2 marks]

TASK 4: DATABASE MODIFICATIONS (15 marks)

Your answers for this task (Task 4) must be placed in the supplied SQL script **T4-brm-mods.sql** **The required changes must be made to the "live" database (the database *after* you have completed tasks 1, 2 and 3, in which other users must be assumed to be active). You MUST not** edit and execute your schema file again. Before completing the work below, please ensure you have completed tasks 1, 2 and 3 above.

In completing this task, you **must**:

- if you need to add new columns, tables or related constraints, follow the naming conventions used in the data models and schema file which have been provided,
- provide column comments for any new columns that you add, and
- correctly manage any transactions used as part of your solution
- as part of your solution, provide appropriate select and desc statements to show the changes you made

(a) BRM would like to determine the status of a quote - whether a quote has been assigned to a job or not (as Y or N). If a quote is not assigned to a job by the preferred start date, the reason for not being converted to a job will be recorded. Until a quote is assigned to a job, the status is set as not assigned.

Add new attributes which will record this requirement. Based on the data which is currently stored in the system, the status attribute must be initialised to the *correct current state of the quote* (i.e. whether there is a job related to the quote or not). You may leave the reason as blank for those not assigned.

Modify the database structure to meet this new requirement.

As part of your solution, provide appropriate select and desc statements to show the changes you have made. Select to show any data changes that have occurred, e.g. `select * from customer;` to show any customer data change, and provide appropriate desc statements to show the changes you have made, e.g. `desc customer;` to show any customer table structural changes.

[4 marks]

(b) Each truck needs regular services. BRM stores the start date and time for which the truck is serviced. Each service may include several service tasks (e.g. oil changes, tire rotations, or brake inspections). The list of the tasks may expand over time. For each service task within a service, BRM records the mechanic who does the task and a free text note (up to 200 characters) which explains the task (only one mechanic is assigned for each service task). Once the service is complete, BRM stores the service end date and time.

Change the database to satisfy this requirement. If you create new tables, you do not need to populate these new tables.

As part of your solution, provide appropriate desc statements to show the changes you have made, e.g. `desc customer;` to show any customer table structural changes.

[11 marks]

TASK 5: PL/SQL (20 marks)

Your commands for this task (Task 5) must be placed in the supplied SQL script **T5-brm-plsql.sql**. **Before completing the work below, please ensure you have completed tasks 1, 2, 3 and 4 above.**

For each of these PL/SQL questions, as part of your answer, you must create a set of SQL commands that will demonstrate the successful operation of your function/trigger/stored procedure (a test harness) - these tests are part of the awarded marks for each question. Place these commands below your function/ trigger/stored procedure definition for each task. **You may do a manual look-up** when writing the test harness.

Ensure your function/trigger/stored procedure definition finishes with a slash(/) followed by a blank line as detailed in the PL/SQL workshop and the applied class materials. In addition, you must provide error/output messages where appropriate.

(a) Create **one** function to return the role of an employee. The function must take the employee number as the input parameter, and return the employee role (either B, T, D, or M).

[2 marks]

(b) From this point onwards, BRM wants to automatically check the integrity of the data inserted into the EMPLOYEE table. You must observe the data model and read the brief carefully to identify the business requirements that you have to handle within the trigger. If any of these rules is breached, the insertion of the new employee must be prevented and an appropriate error message must be displayed. Note that your trigger does not need to handle the update or deletion of an employee. Create **one** trigger to implement this business rule. You may use the function created in (a) as part of your trigger

[8 marks]

(c) Write **one** stored procedure called **prc_entry_job** that handles the job assignment. The procedure must only handle one job assignment at a time.

You must read the brief carefully to identify the business requirements that you have to handle within the procedure. You may use the function created in (a) as part of your procedure.

The procedure requires:

- Nine input parameters (you must **not** change the parameter order)
    1. quote number
    2. driver (employee) id
    3. truck dispatcher (employee) id
    4. truck VIN
    5. trailer code
    6. job cost (can be null if it is the same as the quote cost)
    7. job payment status
    8. pick up date time
    9. job duration (in hours)
- One output parameter which returns the appropriate error/success message related to the booking

[10 marks]

TASK 6: MongoDB (15 marks)

Your answers for this task (Task 6) must be placed in the supplied sql file **T6-brm-json.sql** and the supplied MongoDB script file **T6-brm-mongo.mongodb.js**.

(a) Write an SQL statement in **T6-brm-json.sql** to generate a collection of JSON documents using the following structure/format from the **BRM** tables. Each document in the collection represents a customer and their quotes in the system. The fields in the document represent

- Customer number as the document _id
- Customer full name
- Customer business name, if not supplied, put a "-"
- Customer full address
- Customer contact number
- Statistics for each customer
  - Number of quotes they have made
  - Number of jobs they have agreed upon
  - Total paid job cost (i.e. the customer has made payment for the jobs), if none, put a "-"
  - Total unpaid job cost, if none, put a "-"
- For each quote they have made
  - Quote number
  - Quote prepared date
  - Quote preferred start date
  - Start location
  - End location
  - Quote cost
  - Whether the quote has been assigned to a job or not
  - Job cost; if the job cost is the same as quote cost, use the quote cost; if none (the quote has not been assigned to a job), put a "-"

A sample document is shown below:

```
{
    "_id": 15,
    "customer_name": "Thomas",
    "customer_business": "-",
    "customer_address": "78 Hay Street, Perth, 6003",
    "customer_phone": "0490123009",
    "customer_stats": {
        "number_of_quotes": 3,
        "number_of_jobs": 2,
        "total_paid_jobcost": "$450.00",
        "total_unpaid_jobcost": "$975.00"
    },
    "quotes": [
        {
            "quote_no": 5,
            "quote_prepared_on": "10-May-2026",
            "preferred_start_date": "20-May-2026",
            "start_location": "55 Grenfell Street, Adelaide SA 5000",
```

```
            "end_location": "180 Flinders Street, Adelaide SA 5000",
            "quote_cost": "$450.00",
            "assigned_to_job": "Y",
            "job_cost": "$450.00"
        },
        {
            "quote_no": 31,
            "quote_prepared_on": "22-Jul-2026",
            "preferred_start_date": "25-Jul-2026",
            "start_location": "8 Lion Road, Sydney NSW 2000",
            "end_location": "20 Park Street, Sydney NSW 2000",
            "quote_cost": "$950.50",
            "assigned_to_job": "Y",
            "job_cost": "$975.00"
        },
        {
            "quote_no": 32,
            "quote_prepared_on": "23-Jul-2026",
            "preferred_start_date": "27-Jul-2026",
            "start_location": "6 Lily Street, Canberra ACT 2601",
            "end_location": "9 Crystal Avenue, Melbourne VIC 3001",
            "quote_cost": "$2320.00",
            "assigned_to_job": "N",
            "job_cost": "-"
        }
    ]
}
```


[8 marks]

Write the MongoDB commands for the following questions, 6(b) - 6(d), in the supplied MongoDB script file named **T6-brm-mongo.mongodb.js**.

Do not modify or relocate any of the comments provided in this script. These comments are used to control the automated run of your script during marking. **Altering or moving the supplied comments will prevent your submission from being graded**. You are required to add additional comments as described on page five of this brief.

You are reminded that you may only use MongoDB commands which have been covered in this unit (use your workshop slides as a reference).

(b) Create a new collection and insert all documents generated in 6(a) above into MongoDB.

Provide a drop collection statement right above the create collection statement. You may pick any collection name.

After the documents have been inserted, use an appropriate db.find command to list the full details of all the documents you added. If you inserted more than 20 documents, only 20 will appear in the output - this is a limitation of the Mongo interface we are using and is acceptable.

[1 mark]

(c) List all customers who have made at least 2 quotes and live in *Melbourne*. Display the customer id, full name, address, contact number, number of quotes made, number of jobs agreed upon, total paid job cost and total unpaid job cost.

You must ensure that the query returns some output. If you need to add further data for this task, you must return and add it as part of task 2 and then rerun brm-schema-insert.sql and all tasks (tasks 1, 2, 3 and 4) before running task 6. If this query does not return data, you will be assigned 0 marks.

[2 marks]

(d) A new customer named Patrick Bosse contacted BigRig Movers and was added to the system. Patrick was assigned customer ID 1001.

(i) Write the necessary MongoDB commands to add Patrick Bosse to your MongoDB collection. You may make up other necessary values for Patrick. Use an appropriate db.find command which shows only the details of this member after making the change, so that you can show/confirm the change which was made.

Patrick Bosse (customer ID 1001) then made their first quote with BigRig Movers. The quote number is 2002. The quote is to transport bottled drinks from Adelaide SA to Melbourne VIC and the quote cost is $3200.00. The quote is immediately assigned to a job (with the same cost as the quote cost) and a payment has been made for the job. You may assign any appropriate additional values to insert this quote details. You may do manual calculation for this task if required and you may assume this customer has not made other quotes.

(ii) Write the necessary MongoDB commands to add this new quote to the collection. Use an appropriate db.find command which shows only the details of this member after making the change, so that you can show/confirm the change which was made.

[4 marks]

Submission Requirements

Due Date: **Monday, 8 June 2026 at 11:55 pm**

*Please note that if you need to resubmit, you **cannot** depend on your staff's availability; therefore, please be VERY CAREFUL with your submission.* **It is strongly recommended that you submit several hours before this time to avoid such issues.**

For this assignment, there are seven files you are **required** to submit to Moodle:

- T1-brm-schema.sql
- T2-brm-insert.sql
- T3-brm-dm.sql
- T4-brm-mods.sql
- T5-brm-plsql.sql
- T6-brm-json.sql
- T6-brm-mongo.mongodb.js

Do not zip these files into one zip archive; submit seven independent SQL scripts. These seven files must also exist in your FITGitLab server repo and *show a clear development history* (at least two pushes per file).

Late submission will incur penalties of -5 marks for every 24 hours the submission is late.

Please note we **cannot mark any work on the GitLab Server**; you must ensure that you submit correctly via Moodle, since it is only in this process that you complete the required student declaration, without which work **cannot be assessed**.

IMPORTANT

You must ensure that you submit correctly to Moodle, since it is only in this process that you complete the required student declaration, without which work **cannot be assessed**.
Work on the GitLab server cannot be marked.

It is your responsibility to ENSURE that the files you submit to Moodle are the correct versions. We strongly recommend that you download the files you submitted to Moodle to an empty folder and double-check the contents.

Marking Guide

The submitted code will be assessed against an optimal solution for these tasks. In some tasks where SQL is involved, several alternative approaches are often possible. Such alternatives will be graded based on the code successfully meeting the brief's requirements. If it does, the answer will be accepted and graded appropriately.

Marking Criteria	Items Assessed
TASK 1 DDL 15 marks	
DDL Creation of tables	**Maximum 7 marks - Create table:** ● Marks awarded for correct table DDL ● Marks awarded for correct attributes/data types ● Marks awarded for correct PK definition ● Mark penalty applied if different table/attribute names used than expressed in the supplied data model ● Mark penalty applied if different order of attributes used than expressed in the supplied data model ● No marks awarded if generated schema used
DDL implementation of non-PK database constraints	**Maximum 8 marks - Non-PK Constraints:** ● Marks awarded for correct implementation of non-PK constraints ● Marks awarded for correct use of column comments
TASK 2 Data Insert 10 marks	
Insert of required items test data	**Maximum 5 marks- Insert of data:** ● Marks awarded for correct insert of required data ● Marks awarded for correct management of transactions
Insert of valid test data	**Maximum 5 marks - Valid data inserted:** ● Marks awarded for validity of data inserted ○ meets the requirements expressed in the assignment brief ● Marks awarded for correct management of dates when inserting
Task 3 DML 15 marks	
	Maximum 15 marks - Satisfy brief requirements: ● Marks awarded (a) - (e) for SQL code which meets the expressed requirement ● Mark penalty applied if commit not used appropriately ● Mark penalty applied if date handling and string database lookups not managed correctly

Task 4 Database Modifications 15 marks	
	Maximum 15 marks - Satisfy brief requirements: ● Marks awarded (a) - (b) for SQL code which meets the expressed requirement (including appropriate use of constraints). In making these modifications there must be no loss of existing data or data integrity within the database. ● Mark penalty applied if commit not used appropriately ● Mark penalty applied if column comments not used where required
Task 5 PL/SQL 20 marks	
	Maximum 20 marks - Satisfy brief requirements: ● Marks awarded (a) - (c) for PL/SQL code which meets the expressed requirement. ● Marks awarded for writing a test harness for each question (a) - (c) which includes both successful and failed tests (all errors your code raises must be tested). ● Mark penalty applied if no output messages are provided where appropriate ● Statements which do not execute correctly in Oracle (ie. returns syntax error) will be awarded a maximum of 50% of the available marks less 1 mark. For example, if a question is worth 6 marks and runs with an error in SQL the *maximum* mark awarded will be 2 marks. ● ***Note that the error generated by drop or raise_application_error commands is an expected error and there will be no penalty on this.***
Task 6 Non Relational Database Queries - MongoDB 15 marks	
	Maximum 15 marks - Satisfy brief requirements: ● Maximum of 8 marks awarded for creation of a JSON document which matches the supplied document format ● Marks awarded, as listed, (b) - (d) for MongoDB code which meets the expressed requirement ● Mark penalty applied if field names and predicates (such as "&eq") are not enclosed in double quotes ● Statements which do not execute correctly in MongoDB will be awarded a maximum of 50% of the available marks less 1 mark. For example, if a question is worth 6 marks and runs with an error in MongoDB the *maximum* mark awarded will be 2 marks

Correct Inclusion of Inline Comments 5 marks	
	• Marks awarded for clear, concise, and informative comments throughout your SQL/PL-SQL/MongoDB code • Mark penalty applied if the comment includes words from languages other than English and/or non-ASCII characters
Correct Use of Git 5 marks	
	• Marks awarded for a minimum of fourteen pushes (**two per file**) showing a clear development history of the work for Assignment 2 • Marks awarded for correct Git author details used in pushes • Marks awarded for the use of meaningful commit messages (i.e. not blank or of the form "Push1")
Penalties	
Use of • VIEWs • SET ECHO or SPOOL commands, and/or • PL/SQL	Use of VIEWS, inclusion of SET ECHO/SPOOL/SET SERVEROUTPUT, and/or PL/SQL commands *other than in Task 5*, will result in a ***grade deduction of 10 marks being applied.***
Incorrect file names	If file names do not follow the requirement (i.e., they are changed from the supplied filenames) **or if a zip file is submitted instead of seven individual files**, a ***grade deduction of 5 marks will be applied***.
Late submission	**-5 marks for every 24 hours late or part thereof**

Your mark will be scaled to a maximum of 30, with up to 5 marks added based on your completed Applied assessment tasks (AT7, AT9, AT10, AT11 and AT12).